

React Lab: Conditional Rendering, Lists, Keys, and map()

Hands-on Lab Solution — "bloggerapp" (BookDetails, BlogDetails, CourseDetails)

Objectives

- Explain various ways of conditional rendering
- Explain how to render multiple components
- Define list component
- Explain about keys in React applications
- Explain how to extract components with keys
- Explain React map(), the map() function

Lab Goals

- Implement conditional rendering in React applications

Prerequisites

- Node.js
- NPM
- Visual Studio Code

1. Ways of Conditional Rendering in React

Conditional rendering means showing different UI (or nothing at all) depending on some condition, exactly like conditionals work in plain JavaScript. React does not have special syntax for this — you simply use normal JavaScript inside your component. The most common techniques are:

- if / else statement — return different JSX from different branches (often by returning early).
- Element variables — assign JSX to a variable inside the function, then insert that variable into the returned JSX.
- Ternary operator (condition ? A : B) — an inline expression for choosing between two pieces of JSX.
- Logical && operator (condition && <JSX/>) — renders the JSX only when the condition is true, renders nothing otherwise.
- switch statement — useful when choosing between more than two possible outcomes.
- Immediately Invoked Function Expression (IIFE) — ((() => { ... }())) lets you run multi-line logic directly inside JSX.
- Returning null — a component can return null to render nothing at all.

This lab demonstrates each technique using one of the three bloggerapp components.

2. Rendering Multiple Components

A React page is normally composed of several smaller components rendered together inside a parent component, such as App. Multiple components can be placed side by side in JSX (they just need to be wrapped in a single parent element or a Fragment), and which ones actually appear can be controlled with the conditional

rendering techniques above, buttons/state, or by rendering a whole collection of components from an array using `map()`.

```
return (  
  <div className="App">  
    <BookDetails show={true} />  
    <BlogDetails show={true} />  
    <CourseDetails show={true} />  
  </div>  
)  
);
```

3. List Component

A list component is a component whose job is to take a collection (an array) of data and render one UI element for every item in that collection, usually inside a `<ul>/<ol>` or a series of `<div>`s. In React this is done with the array's `map()` method, which transforms an array of data into an array of JSX elements that React can render directly.

```
const books = [  
  { id: 1, title: 'The Alchemist', author: 'Paulo Coelho' },  
  { id: 2, title: 'Atomic Habits', author: 'James Clear' },  
  { id: 3, title: 'Sapiens', author: 'Yuval Noah Harari' }  
];  
  
<ul>  
  {books.map((book) => (  
    <li key={book.id}>{book.title} by {book.author}</li>  
  ))}  
</ul>
```

4. Keys in React Applications

A key is a special string attribute you must include when creating a list of elements. Keys help React identify which items have changed, been added, or been removed between renders, so it can update the DOM efficiently instead of re-rendering the entire list. Keys only need to be unique among siblings in the same array, not globally unique across the whole app.

- Always use a stable, unique identifier from your data (like an id) as the key, not the array index, whenever the list can be reordered, filtered, or have items inserted/removed.
- Using the array index as a key is acceptable only for static lists that never change order.
- Omitting keys or using non-unique keys causes React console warnings and can lead to subtle rendering bugs (e.g. stale input values after reordering).

```
// Not recommended when the list can change order:  
{books.map((book, index) => <li key={index}>{book.title}</li>)}  
  
// Recommended: use a stable id from the data itself  
{books.map((book) => <li key={book.id}>{book.title}</li>)}
```

5. Extracting Components with Keys

When a list item becomes complex, it's common to extract it into its own component. The important rule is that the key belongs on the element created in the loop (i.e. where the component is called/invoked), not inside the extracted component itself — keys are only meaningful in the context of the surrounding array.

```
// Extracted list-item component – no 'key' prop is read inside it
function Book({ title, author }) {
  return <li>{title} by {author}</li>;
}

// The key is placed where Book is used inside the map(), not inside Book itself
<ul>
  {books.map((book) => (
    <Book key={book.id} title={book.title} author={book.author} />
  ))}
</ul>
```

6. React Map / the map() Function

`Array.prototype.map()` is a standard ES6 array method (not a React-specific API), but it is the standard way React apps turn arrays of data into arrays of JSX elements. `map()` takes a callback function, applies it to every element of the source array, and returns a brand-new array of the same length — it never mutates the original array. Because JSX can render arrays of elements directly, wrapping a `map()` call in curly braces inside JSX is enough to render a whole list.

```
const courseNames = courses.map((course) => course.name);
// courseNames is a new array: ['React Fundamentals', 'Advanced JavaScript', ...]
```

Lab Steps

Step 1: Create the React application

```
npx create-react-app bloggerapp
cd bloggerapp
npm start
```

Open the `bloggerapp` folder in Visual Studio Code. Inside the `src` folder, create three new files: `BookDetails.js`, `BlogDetails.js`, and `CourseDetails.js`.

Step 2: `BookDetails` — conditional rendering with `if / else` + returning `null`

`BookDetails` renders a list of books using `map()` with a key on each `<li>`. If the `show` prop is `false`, the component returns `null` via an early `if` statement, so nothing is rendered at all.

src/BookDetails.js

```
import React from 'react';

function BookDetails(props) {
  const books = [
    { id: 1, title: 'The Alchemist', author: 'Paulo Coelho' },
    { id: 2, title: 'Atomic Habits', author: 'James Clear' },
    { id: 3, title: 'Sapiens', author: 'Yuval Noah Harari' }
  ];
```

```
// Conditional rendering #1: if / else (return null hides the component)
if (!props.show) {
  return null;
}

return (
  <div>
    <h2>Book Details</h2>
    <ul>
      {books.map((book) => (
        <li key={book.id}>{book.title} by {book.author}</li>
      ))}
    </ul>
  </div>
);
}

export default BookDetails;
```

Step 3: BlogDetails — conditional rendering with a ternary operator + element variable

BlogDetails builds an element variable, `listOrMessage`, using a ternary expression, and then inserts that variable into the returned JSX.

src/BlogDetails.js

```
import React from 'react';

function BlogDetails(props) {
  const blogs = [
    { id: 1, title: 'Getting Started with React' },
    { id: 2, title: 'Understanding useState' },
    { id: 3, title: '10 Tips for Clean Code' }
  ];

  // Conditional rendering #2: ternary operator, stored as an element variable
  const listOrMessage = props.show ? (
    <ul>
      {blogs.map((blog) => (
        <li key={blog.id}>{blog.title}</li>
      ))}
    </ul>
  ) : (
    <p>Blog details are hidden.</p>
  );

  return (
    <div>
      <h2>Blog Details</h2>
      {listOrMessage}
    </div>
  );
}

export default BlogDetails;
```

Step 4: CourseDetails — conditional rendering with the && operator + extracted list-item component

CourseDetails uses the logical && operator directly inline in JSX. It also demonstrates extracting the list item into its own component, CourseItem, with the key placed on the CourseItem element inside map() rather than inside CourseItem itself.

src/CourseDetails.js

```
import React from 'react';

// Extracted list-item component (does not need to know about 'key')
function CourseItem({ name }) {
  return <li>{name}</li>;
}

function CourseDetails(props) {
  const courses = [
    { id: 1, name: 'React Fundamentals' },
    { id: 2, name: 'Advanced JavaScript' },
    { id: 3, name: 'Node.js for Beginners' }
  ];

  return (
    <div>
      <h2>Course Details</h2>
      { /* Conditional rendering #3: logical && operator */ }
      { props.show && (
        <ul>
          { courses.map((course) => (
            <CourseItem key={course.id} name={course.name} />
          )) }
        </ul>
      ) }
      { !props.show && <p>Course details are hidden.</p> }
    </div>
  );
}

export default CourseDetails;
```

Step 5: App.js — conditional rendering with switch + an IIFE, rendering all three components

App.js keeps track of which category is selected in state, and uses a switch statement to build an element variable (content) that decides which single component to highlight, while still always rendering all three components together beneath it (demonstrating 'render multiple components'). An IIFE is used to compute a small derived label inline as a fourth conditional-rendering technique.

src/App.js

```
import React, { useState } from 'react';
import BookDetails from './BookDetails';
import BlogDetails from './BlogDetails';
import CourseDetails from './CourseDetails';

function App() {
  const [selected, setSelected] = useState('books');
```

```

// Conditional rendering #4: switch statement + element variable
let content;
switch (selected) {
  case 'books':
    content = <BookDetails show={true} />;
    break;
  case 'blogs':
    content = <BlogDetails show={true} />;
    break;
  case 'courses':
    content = <CourseDetails show={true} />;
    break;
  default:
    content = <p>Select a category above.</p>;
}

return (
  <div className="App">
    <h1>Blogger App</h1>

    <button onClick={() => setSelected('books')}>Books</button>
    <button onClick={() => setSelected('blogs')}>Blogs</button>
    <button onClick={() => setSelected('courses')}>Courses</button>

    {/* Conditional rendering #5: Immediately Invoked Function Expression (IIFE) */}
    <h3>
      {(() => {
        if (selected === 'books') return 'Showing: Books';
        if (selected === 'blogs') return 'Showing: Blogs';
        return 'Showing: Courses';
      })()}
    </h3>

    {content}

    <hr />
    <h2>All Sections</h2>
    <BookDetails show={true} />
    <BlogDetails show={true} />
    <CourseDetails show={true} />
  </div>
);
}

export default App;

```

Step 6: Render with ReactDOM

src/index.js

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);

```

Step 7: Run the application

```
npm start
```

This opens the app at <http://localhost:3000>.

Expected Output

```
Blogger App
[Books] [Blogs] [Courses]

Showing: Books

Book Details
The Alchemist by Paulo Coelho
Atomic Habits by James Clear
Sapiens by Yuval Noah Harari

All Sections

Book Details
The Alchemist by Paulo Coelho
Atomic Habits by James Clear
Sapiens by Yuval Noah Harari

Blog Details
Getting Started with React
Understanding useState
10 Tips for Clean Code

Course Details
React Fundamentals
Advanced JavaScript
Node.js for Beginners
```

Clicking Blogs or Courses updates the “Showing:” heading and the highlighted section, driven by the switch statement in App.js.

Summary

- This lab used five different conditional-rendering techniques across the three components: if/else with a null return (BookDetails), a ternary operator stored in an element variable (BlogDetails), the logical && operator (CourseDetails), a switch statement (App.js), and an IIFE (App.js).
- Multiple components (BookDetails, BlogDetails, CourseDetails) are composed together inside the parent App component.
- Each component acts as a list component, using `Array.prototype.map()` to turn an array of data into an array of JSX `<li>` elements.
- Every element produced inside a `map()` call is given a unique key prop (an id from the data), which lets React efficiently track additions, removals, and reordering.
- CourseDetails demonstrates extracting a list item into its own component (CourseItem), with the key kept on the outer element inside `map()` rather than inside the extracted component.